



Programming Java

Networking

Incheon Paik



Contents

- Internet Addresses
- Server Sockets and Sockets
- Datagram Sockets and Packets
- Uniform Resource Locators (URL)
- The Java Remote Method Invocation (RMI)



Internet Addresses

- Transmission Control Protocol (TCP) : obtain reliable, sequenced data exchange.
- User Datagram Protocol (UDP) : obtain a more efficient, best-effort delivery.

GetByName() Method

static InetAddress getByName (String hostName) throws UnknownHostException

getAllByName() Method

static InetAddress[] getAllByName (String hostName) throws UnknownHostException

getLocalHost() Method

static InetAddress getLocalHost() throws UnknownHostException

```
import java.net.*;
```

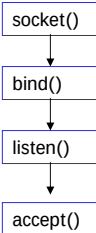
```
class InetAddressDemo {  
    public static void main(String args[]) {  
        try {  
            InetAddress ias[] =  
                InetAddress.getAllByName(args[0]);  
            for (int i = 0; i < ias.length; i++) {  
                System.out.println(ias[i].getHostName());  
                System.out.println(ias[i].getHostAddress());  
                byte bytes[] = ias[i].getAddress();  
                for (int j = 0; j < bytes.length; j++) {  
                    if (j > 0)  
                        System.out.print(".");  
                    if (bytes[j] >= 0)  
                        System.out.print(bytes[j]);  
                    else  
                        System.out.print(bytes[j] + 256);  
                }  
                System.out.println("");  
            }  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

<http://java.sun.com/j2se/1.5.0/docs/api/java/net/InetAddress.html>



Socket Call for Connection-Oriented Protocol

Server



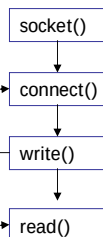
Blocks until connection from client

read()

Process request

write()

Client



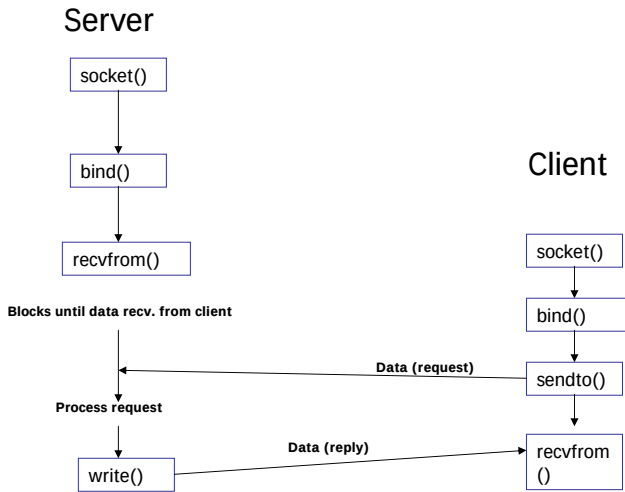
connection establishment

Data (request)

Data (reply)



Socket Call for Connectionless Protocol



Server Sockets and Sockets

ServerSocket Constructor

ServerSocket(int port) throws IOException

accept() Method

Socket accept() throws IOException

close() Method

void close() throws IOException

Socket Class

*Socket(String hostName, int port) throws
UnknownHostException, IOException*

getInputStream(), getOutputStream Method

*InputStream getInputStream() throws IOException
OutputStream getOutputStream() throws IOException*

close()

void close() throws IOException

<http://java.sun.com/j2se/1.5.0/docs/api/java/net/ServerSocket.html>

<http://java.sun.com/j2se/1.5.0/docs/api/java/net/Socket.html>



Server Sockets and Sockets

```
import java.io.*;
import java.net.*;
import java.util.*;

class ServerSocketDemo {
    public static void main(String args[]) {
        try {

            // Get Port
            int port = Integer.parseInt(args[0]);
            Random random = new Random();
            //Create Server Socket
            ServerSocket ss = new ServerSocket(port);
            //Create Infinite Loop
            while(true) {
                //Accept Incoming Requests
                Socket s = ss.accept();

                //Write Result to Client
                OutputStream os = s.getOutputStream();
                DataOutputStream dos = new
DataOutputStream(os);
                dos.writeInt(random.nextInt());

                //Close socket
                s.close();
            }
        } catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

```
class SocketDemo {
    public static void main(String args[]) {
        try {
            //Get Server and Port
            String server = args[0];
            int port = Integer.parseInt(args[1]);
            //Create socket
            Socket s = new Socket(server, port);
            //Read random number from server
            InputStream is = s.getInputStream();
            DataInputStream dis = new
DataInputStream(is);
            int i = dis.readInt();
            //Display Result
            System.out.println(i);
            //Close Socket
            s.close();
        }
        catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

Running :

```
% java ServerSocketDemo 4321
% java SocketDemo 127.0.0.1 4321
```

Java

7

Networking



Datagram Sockets and Packets

- UDP does not guarantee reliable, sequenced data exchange, and therefore requires much less overhead.

DatagramPacket Constructor

```
DatagramPacket(byte buffer[], int size)
DatagramPacket(byte buffer[], int size, InetAddress ia,
int port)
```

DatagramSocket() Method

```
DatagramSocket() throws SocketException
DatagramSocket(int port) throws SocketException
```

receive() Method

```
void receive(DatagramPacket dp) throws IOException
```

send() Method

```
void send(DatagramPacket dp) throws IOException
```

close() Method

```
void close()
```

<http://java.sun.com/j2se/1.5.0/docs/api/java/net/DatagramPacket.html>

8

Java

Networking

Datagram Sockets and Packets

```
class DatagramReceiver {
    private final static int BUFSIZE = 20;
    public static void main(String args[]) {
        try {
            //Obtain port
            int port = Integer.parseInt(args[0]);

            //Create a DatagramSocket object for the port
            DatagramSocket ds = new DatagramSocket(port);

            //Create a buffer to hold incoming data
            byte buffer[] = new byte[BUFSIZE];

            //Create infinite loop
            while(true) {
                //Create a datagram packet
                DatagramPacket dp =
                    new DatagramPacket(buffer, buffer.length);

                //Receive data
                ds.receive(dp);

                //Get data from the datagram packet
                String str = new String(dp.getData());

                //Display the data
                System.out.println(str);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
class DatagramSender {
    public static void main(String args[]) {
        try {
            // Create destination Internet address
            InetAddress ia =
                InetAddress.getByName(args[0]);
            // Obtain destination port
            int port = Integer.parseInt(args[1]);

            // Create a datagram socket
            DatagramSocket ds = new DatagramSocket();

            //Create a datagram packet
            byte buffer[] = args[2].getBytes();
            DatagramPacket dp =
                new DatagramPacket(buffer, buffer.length,
                    ia, port);
            // Send the datagram packet
            ds.send(dp);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Running :

% java DatagramReceiver 4321

% java DatagramSender localhost 4321 Message

Uniform Resource Locators (URL)

URL

Protocol://host:/port/file

openStream() Method

InputStream() throws IOException

URL Constructor

*URL(String protocol, String host, int port, String file)
throws MalformedURLException*
*URL(String protocol, String host, String file) throws
MalformedURLException*
URL(String urlString) throws MalformedURLException

getFile(), getHost(), getPort(), and getProtocol() Methods

*String getFile()
String getHost()
int getPort()
String getProtocol()*

Refer <http://java.sun.com/j2se/1.5.0/docs/api/java/net/URL.html>



URL Demo Example

```
class URLDemo {
    public static void main(String args[]) {
        try {

            // Obtain URL
            URL url = new URL(args[0]);

            // Obtain input stream
            InputStream is = url.openStream();

            // Read and display data from URL
            byte buffer[] = new byte[1024];
            int i;
            while((i = is.read(buffer)) != -1) {
                System.out.write(buffer, 0, i);
            }
        }
        catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Run :

```
java URLDemo http://www.u-aizu.ac.jp
```

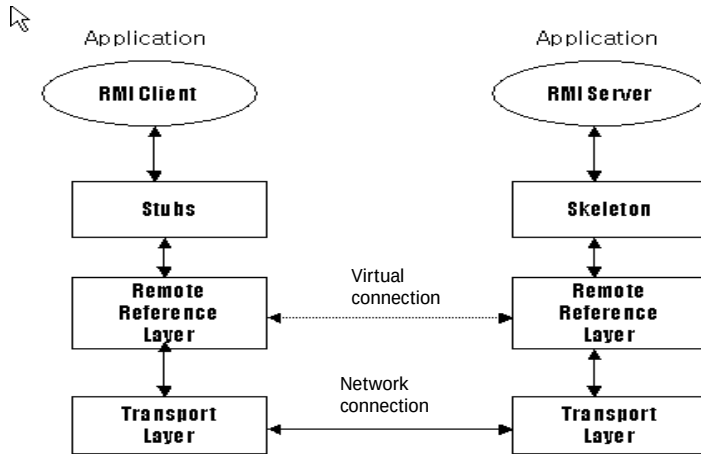


Introduction to RMI

- ☐ Distributed Processing on Network
- ☐ Define of Remote Interface
- ☐ Object Serialization
- ☐ java.rmi and java.rmi.server
- ☐ Create Stub and Skeleton



RMI Architecture



Example of RMI Implementation

- ☐ Interface Definition
- ☐ Implementation Class and Compilation
- ☐ Creation of Stub and Skeleton using `rmic`
- ☐ Creation of Server Application and Compilation
- ☐ RMI Registry and Start Server Program
- ☐ Creation of Client Program and Compilation
- ☐ Test Client



Hello Example : RMI

Interface

```
package examples.hello;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Hello extends Remote {
    String sayHello() throws RemoteException;
}
```



Hello Example : RMI

Implementation

```
package examples.hello;

import java.rmi.Naming;
import java.rmi.RemoteException;
import java.rmi.RMISecurityManager;
import java.rmi.server.UnicastRemoteObject;

public class HelloImpl extends UnicastRemoteObject
    implements Hello {

    public HelloImpl() throws RemoteException {
        super();
    }

    public String sayHello() {
        return "Hello World!";
    }

    public static void main(String args[]) {
        // Create and install a security manager
        if (System.getSecurityManager() == null) {
            System.setSecurityManager(new
                RMISecurityManager());
        }
    }
}
```

```
try {
    HelloImpl obj = new HelloImpl();
    // Bind this object instance to the
    name "HelloServer"
    Naming.rebind("HelloServer", obj);
    System.out.println("HelloServer bound
in registry");
} catch (Exception e) {
    System.out.println("HelloImpl err: " +
e.getMessage());
    e.printStackTrace();
}
}
```

Compile & Skeleton Creation :

```
% javac examples/hello/Hello.java
% javac examples/hello/HelloImpl.java
% rmic examples.hello.HelloImpl
```




Hello Example : RMI

A Client Application

```
package examples.hello;

import java.rmi.Naming;
import java.rmi.RemoteException;

public class HelloClient {

    public static void main(String args[]) {
        String message = "Hello: This is my test message";

        // "obj" is the identifier that we'll use to refer
        // to the remote object that implements the "Hello"
        // interface
        Hello obj = null;

        try {
            obj = (Hello)Naming.lookup("//" + "/HelloServer");
            message = obj.sayHello();
        } catch (Exception e) {
            System.out.println("HelloClient exception: " + e.getMessage());
            e.printStackTrace();
        }

        System.out.println("Message = " + message);
    } // end of main
} // end of HelloClient
```



Hello Example : RMI

File "policy"

```
grant {
    // Allow everything for now
    permission java.security.AllPermission;
};
```

Start Registry Server & Run Server and Client

```
% rmiregistry &
% java -Djava.security.policy=policy examples.hello.HelloImpl
% javac examples/hello/HelloClient.java
% java [-Djava.security.policy=policy] examples.hello.HelloClient
```

Please ensure there is the "policy" file
in the current directory



Exercise

- **Step 1, 2 (A BMI Server and a Simple Chatting Program)**

Use the ServerSocket and Socket class
Slide # 4-5

- **Step 3 (RMI Example)**

Refer Slide #10-17